



2.1 Introduction to Buffers

In Node.js, buffers are a special type of object that can store raw binary data. A buffer represents a chunk of memory - typically RAM - allocated in your computer. Once set, the size of a buffer cannot be changed.

A buffer stores bytes. A byte is a sequence of eight bits. Bits are the most basic unit of storage on your computer, they can hold the value of either 0 or 1.

Node.js exposes the **Buffer** class in the global scope (you don't need to import or require it like other modules). With this API, you get a series of functions and abstractions to manipulate raw binaries.

Syntax:

```
const buf = Buffer.alloc(10); // Allocates a buffer of 10 bytes.
```

Generally, **Buffer refers to the particular memory location in memory**. Buffer and array have some similarities, but the difference is array can be any type, and it can be resizable. Buffers only deal with binary data, and it cannot be resizable. Each integer in a buffer represents a byte. `console.log()` function is used to print the Buffer instance.

Buffer Methods:

No	Method	Description
1	Buffer.alloc(size)	It creates a buffer and allocates size to it.
2	Buffer.from(initialization)	It initializes the buffer with given data.
3	Buffer.write(data)	It writes the data on the buffer.
4	toString()	It read data from the buffer and returned it.
5	Buffer.isBuffer(object)	It checks whether the object is a buffer or not.
6	Buffer.length	It returns the length of the buffer.
7	Buffer.copy(buffer, subsection size)	It copies data from one buffer to another.
8	Buffer.slice(start, end=buffer.length)	It returns the subsection of data stored in a buffer.
9	Buffer.concat([buffer,buffer])	It concatenates two buffers.

Creating buffers

Buffers are created using these three methods:

- Buffer.from()
- Buffer.alloc()
- Buffer.allocUnsafe()

In the past, buffers were created using the Buffer class constructor (e.g., new Buffer()). This syntax is now deprecated.

Buffer.from(): This method is the most straightforward way to create a buffer. It accepts a string, an array, an **ArrayBuffer**, or another buffer instance. Depending on which params you pass, **Buffer.from()** will create a buffer in a slightly different way.

When passing a string, a new buffer object will be created containing that string. By default, it will parse your input using utf-8 as the encoding .

Name of the Faculty: Gopal Paul

Designation and Department: Assistant Professor, Computational Science

Brainware University, Kolkata



```
// Creates a new buffer with the string 'heya!'
// If no encoding is passed in the second parameter, defaults to 'utf-8'.
Buffer.from('heya!')
// Creates the same buffer as the above, but passes 'heya!' as a hex encoded
stringBuffer.from('6865796121', 'hex')
```

When passing a buffer to **Buffer.from()**, Node.js will copy that buffer into the current one. The new buffer is allocated in a different area of memory, so you can modify it independently:

```
const buffer1 = Buffer.from('cars')
// Creates a buffer from `buffer1`
const buffer2 = Buffer.from(buffer1)
// Modify `buffer2`
buffer2[0] = 0x6d
// 0x6d is the letter "m"
console.log(buffer1.toString()) // --> "cars"
console.log(buffer2.toString()) // --> "mars"
```

Buffer.alloc(): The **.alloc()** method is useful when you want to create empty buffers, without necessarily filling them with data. By default, it accepts a number and returns a buffer of that given size filled with 0s:

```
Buffer.alloc(6)
// --> <Buffer 00 00 00 00 00 00>
// Creates a buffer of size 1 filled with 0s (<Buffer 00>)
const buff = Buffer.alloc(1);
// Fill the first (and only) position with
contentbuff[0] = 0x78
// 0x78 is the letter "x"
console.log(buff.toString('utf-8'));
// --> 'x'
```

Can also fill the buffer with other content than 0 and a given encoding:

```
Buffer.alloc(6, 'x', 'utf-8') // --> <Buffer 78 78 78 78 78 78>
```

Buffer.allocUnsafe(): With **.allocUnsafe()**, the process of sanitizing and filling the buffer with 0s is skipped. The buffer will be allocated in a area of memory that may contain old data (that's where the "unsafe" part comes from). For example, the following code will most likely always print some random pieces of data every time you run it:

```
// Allocates a random area of memory with size 10000
// Does not sanitizes it (fill with 0) so it may contain old data
const buff = Buffer.allocUnsafe(10000)
// Prints loads of random data
console.log(buff.toString('utf-8'))
```



A good use case for **.allocUnsafe()** is when you are copying a buffer that was safely allocated. Since you will completely overwrite the copied buffer, all the old bytes will be replaced by predictable data:

```
// Creates a buffer from a string
const buff = Buffer.from('hi, I am a safely allocated buffer')
// Creates a new empty buffer with `allocUnsafe` of the same
// length as the previous buffer. It will be initially filled with old data.
const buffCopy = Buffer.allocUnsafe(buff.length)
// Copies the original buffer into the new, unsafe buffer.
// Old data will be overwritten with the bytes
// from 'hi, I am a safely allocated buffer' string.
buff.copy(buffCopy)
console.log(buffCopy.toString())
// --> 'hi, I am a safely allocated buffer'
```

Writing to buffers

The way to write data into buffers is using **Buffer.write()**. By default, it will write a string encoded in **utf-8** with no offset (starts writing from the first position of the buffer). It returns a number, which is the number of bytes that were written in the buffer:

```
const buff = Buffer.alloc(9)
buff.write('hey there')
// returns 9 (number of bytes written)
// If you write more bytes than the buffer supports,
// your data will truncated to fit the buffer.
buff.write('hey christopher')
// returns 9 (number of bytes written)
console.log(buff.toString())
// -> 'hey chris'
```

Keep in mind that not all characters occupy a single byte in the buffer (!):

```
const buff = Buffer.alloc(2)
// The copyright symbol ('©') occupies two bytes,
// so the following operation will completely fill the buffer. buff.write('©')
// returns 2
// If the buffer is too small to store the character, it will not write it.
const tinyBuff = Buffer.alloc(1)
tinyBuff.write('©')
// returns 0 (nothing was written)
console.log(tinyBuff)
// --> <Buffer 00> (empty buffer)
```

Also notice that 2 is not the highest number of bytes a character can have. For example, the **utf-8** encoding type supports characters with up to 4 bytes. Since you cannot modify the



size of the buffer, you always need to be mindful of what you are writing and how much space it will take (size of the buffer vs. size of your content).

Example Of Buffer:

```
// Filename: index.js

// Different Method to create Buffer
const buffer1 = Buffer.alloc(100);
const buffer2 = Buffer.from('BWU');
const buffer3 = Buffer.from([1, 2, 3, 4]);

// Writing data to Buffer
buffer1.write("Happy Learning");

// Reading data from Buffer
const a = buffer1.toString('utf-8');
console.log(a);

// Check object is buffer or not
console.log(Buffer.isBuffer(buffer1));

// Check length of Buffer
console.log(buffer1.length);

// Copy buffer
const bufferSrc = Buffer.from('ABC');
const bufferDest = Buffer.alloc(3);
bufferSrc.copy(bufferDest);

const Data = bufferDest.toString('utf-8');
console.log(Data);

// Slicing data
const bufferOld = Buffer.from('Node For Server');
const bufferNew = bufferOld.slice(0, 4);
console.log(bufferNew.toString());

// concatenate two buffer
const bufferOne = Buffer.from('Happy Learning ');
const bufferTwo = Buffer.from('With GFG');
const bufferThree = Buffer.concat([bufferOne, bufferTwo]);
```



```
console.log(bufferThree.toString());
```

2.2 Node.js modules

Understanding Modules in Node.js

Modules in Node.js are reusable units of code that can be imported and shared across various parts of your application. These modules can contain functions, objects, or variables that are exported from one file and imported into another, promoting modularity and reusability.

What Are Modules in Node.js?

In Node.js, a module is similar to a JavaScript library – it's a collection of related functionality that can be included in your application. Every file in Node.js is treated as a module, capable of exporting functions or objects for use elsewhere in the application. This approach allows developers to break down their code into smaller, more manageable pieces, which fosters better organization and code reusability.

How Node.js Modules Operate

Each module in Node.js runs in its own isolated scope. This means that any variables, functions, or classes defined inside a module are not accessible globally unless they are explicitly exported. This isolation helps prevent conflicts in the global namespace and contributes to cleaner, more maintainable code. By exporting only the necessary functionality, developers can ensure that only the relevant parts of a module are accessible to other parts of the application, promoting better encapsulation and security.

At its core, a Node.js module is an object that includes several important properties:

- **exports:** This is the object that a module uses to expose functionality to other modules.
- **require():** This is a function used to import modules into the current module.
- **module:** This represents the module itself, providing details about the current module.

Node.js includes three types of modules:

1. Core Modules
2. Local Modules
3. Third Party Modules

2.2.1 Node core modules

Core modules are built-in modules that come bundled with Node.js. They cover a wide range of functionalities, from file system operations to network communication. You can use them without needing to install external packages, making them a fundamental part of Node.js development.

Core modules come with Node.js and don't need to be installed. They provide low-level functionality and helper methods.

Core modules allow Node.js to work with the filesystem, networking, binary data, streams, spawn external processes, parse query strings, file paths, and URLs, and perform other helpful tasks such as creating HTTP(S) agents/clients and servers.

List of main core modules:

- **fs:** module to work with the file system, files, and folders.
- **path:** module to parse file system paths across platforms.

Name of the Faculty: Gopal Paul

Designation and Department: Assistant Professor, Computational Science

Brainware University, Kolkata



- querystring: module to parse query string data.
- net: module to work with networking for various protocols.
- stream: module to work with data streams.
- events: module to implement event emitters (Node observer pattern).
- child_process: module to spawn external processes.
- os: module to access OS-level information including platform, number of CPUs, memory, uptime, etc.
- url: module to parse URLs.
- http: module to make requests (client) and accept requests (server).
- https: module to do the same as http only for HTTPS.
- util: various utilities, including promises which turn any standard Node core method into a promise-base API.
- assert: module to perform assertion-based testing.
- crypto: module to encrypt and hash information.

Include core modules

There is no need to install or download core modules.

To include them in your application, use **require()** and enter the module/package name as the name string.

Syntax:

```
const ref_name=require("module_name")
```

For example, the following code imports a package named http.

```
const http = require("http");  
//replace `http` with the core module you want to use
```

The package is in

- the node_modules folder in the root of the project if it's an installed npm package
- the system folder if it's a core Node module (location depends on your OS and how you installed Node)

➤ **What is the Path module?**

Path Module in Node.js is used to manipulate the path of files.

To include the path module, we use the require() method:

```
const path = require("path");
```

✓ **Get the file name - path.basename(path[, ext]):** We can use the path.basename() function to get the file name.

```
console.log(path.basename("public/image/cat.png")); // cat.png  
console.log(path.basename("public/image/cat.png", ".png")); // cat
```

Example: We check the current file name

```
// Base file name  
console.log(path.basename(__filename)); // hello.js
```

Name of the Faculty: Gopal Paul

Designation and Department: Assistant Professor, Computational Science

Brainware University, Kolkata



- ✓ **Get the folder path of the file - `path.dirname(path)`:** `path.dirname()` helps get the path to the file's folder.

Example:

```
console.log(path.dirname(__filename));  
// Result(Current directory): /home/user/dir  
console.log(path.dirname("/public/image/logo.png"));  
// Result: /public/image
```

- ✓ **Get the file extension - `path.extname(path)` :** Get the file extension value, but there are some special cases.

Example:

```
console.log(path.extname(__filename)); // .js  
console.log(path.extname("cat.html")); // .html  
console.log(path.extname("cat.md")); // .md  
console.log(path.extname("cat.")); // .  
console.log(path.extname("cat")); // (blank)  
console.log(path.extname(".cat")); // (blank)
```

- ✓ **Get Path String from an Object - `path.format(object)` :** This function will return a path with an Object as a value. This object has five elements, including:

- root
- dir
- base
- ext
- name

Case 1: If dir and base are provided, the result will be dir + base.

```
console.log(  
  path.format({  
    dir: "/home/user/dir",  
    base: "path.js",  
  })  
);  
// /home/user/dir/path.js
```

Case 2: root will be used if dir is not available

```
console.log(  
  path.format({  
    root: "/",  
    base: "path.js",  
  })  
);  
// /path.js
```

Case 3: name + ext is used if there is no base

Name of the Faculty: Gopal Paul

Designation and Department: Assistant Professor, Computational Science

Brainware University, Kolkata



```
console.log(
  path.format({
    dir: "/",
    name: "path",
    ext: ".js",
  })
);
// /path.js
```

Case 4: If dir and root are used, they will be combined.

```
console.log(
  path.format({
    root: "/",
    dir: "/home/user/dir",
    base: "path.js",
    ext: ".js",
    name: "path",
  })
);
// /home/user/dir/path.js
```

Case 5: If only base is available, it will return base.

```
console.log(
  path.format({
    base: "path.js",
  })
);
// path.js
```

✓ **Convert path to an object - *path.parse(path)*** : This function is opposite to the path.format function.

Example: Log current path object

```
// Create path object
console.log(path.parse(__filename));
/*
{
  root: '/',
  dir: '/home/user/dir',
  base: 'path.js',
  ext: '.js',
  name: 'path'
}
```



```
*/
```

✓ **Concatenate paths - `path.join([path1][, path2][, ...])`** : To concatenate paths, we use `path.join()`.

```
console.log(path.join(__dirname, "test", "hello.html"));
```

```
// /home/user/dir/test/hello.html
```

✓ **Check absolute path - `path.isAbsolute(path)`** : `path.isAbsolute()` checks whether the path is an absolute path.

```
console.log(path.isAbsolute("/foo/bar")); // true
```

```
console.log(path.isAbsolute("/baz/..")); // true
```

```
console.log(path.isAbsolute("hello/")); // false
```

```
console.log(path.isAbsolute(".")); // false
```

✓ **Separate path - `path.sep`** : We use `path.sep` to separate path.

```
console.log("foo/bar/baz".split(path.sep)); // [ 'foo', 'bar', 'baz' ]
```

➤ **What is Node.js File system module?**

In our computer, data is organized and accessed through a file system. The Node.js file system module allows us to interact with the file system.

fs handles file system operations such as reading to and writing from files. There are synchronous and asynchronous methods in the library.

To include the File System module, we can use the `require()` method:

```
const fs = require("fs");
```

✓ **Read file**

To read the content of a file using NodeJS, we can use the `readFile()` and `readFileSync()` methods.

`readFile()` : The **`fs.readFile()`** method allows us to read the content of a file asynchronously, so the execution of JavaScript code will continue down the line without waiting for the method to finish.

It takes three parameters:

- A relative path to the file we want to read
- A character encoding
- A callback function that will fire when this is completed

```
const fs = require("fs");
fs.readFile("./docs/blog.txt", "utf8", (err, data) => {
  if (err) throw err;
  console.log(data);
});
// Hello world
```

Note: If we don't add `utf8`, when we run the file, it will return a buffer, which is a data package.

To see our string data in text format, we can use the `toString()` method:

Name of the Faculty: Gopal Paul

Designation and Department: Assistant Professor, Computational Science

Brainware University, Kolkata



```
const fs = require("fs");
fs.readFile("./docs/blog.txt", (err, data) => {
  if (err) throw err;
  console.log(data.toString());
});
// Result: Hello world
```

readFileSync(): The `readFileSync()` method reads the content of a file synchronously, so the JavaScript code execution will be stopped until the method is finished.

It has two parameters:

- A relative path to the file we want to read (string type)
- Options: The encoding format of the file content to read with (string or object type)

```
const fs = require("fs");
const data = fs.readFileSync("./docs/blog.txt", "utf8");
console.log(data);
```

Write file

We use the `fs.writeFile()` method to write a file. It takes three arguments:

- The first argument is a relative path to the file we want to write to
- The second argument is a text we want to write to this file
- The third argument is a callback function

For example, we'd like to replace "Hello world" with "Hello there":

```
const fs = require("fs");
fs.writeFile("./docs/blog.txt", "Hello there", () => {
  console.log("file was changed");
});
// Result: file was changed
```

The `fs.writeFile()` asynchronously writes data to the file, replacing it if it already exists. If the file does not exist, a new file will be created.

Append file

Because `fs.writeFile()` overwrite data, if we want to add more to a file, we need to use `fs.appendFile()`.

```
const fs = require("fs");
const path = require("path");

// Create and write to file
fs.writeFile("./docs/blog.txt", "Hello World!", (err) => {
  if (err) throw err;
  console.log("Writing file ...");

  // File append
  fs.appendFile("./docs/blog.txt", "Learning Nodejs", (err) => {
```



```
if (err) throw err;
console.log("Appending file ...");
});
});
```

Rename file

We use `fs.rename()` to rename a file. It takes two arguments:

- The first argument is a relative path to the file we want to rename
- The second argument is a relative path with a new name
- The third parameter is a callback function

```
// Rename file
fs.rename("./docs/blog.txt", "./docs/newblog.txt", (err) => {
  if (err) throw err;
  console.log("File renamed...");
});
```

Delete file

We use the `fs.unlink()` method to delete a file. It takes two arguments:

- The first argument is a relative path to the file we want to delete
- The second argument is a callback function

```
const fs = require("fs");
fs.unlink("./docs/blog2.txt", (err) => {
  if (err) {
    console.log(err);
  }
  console.log("file deleted");
});
// Result: file deleted
```

Check if a file exists

We can check whether a file or a directory exists before deleting it with the `fs.existsSync()` method.

```
const fs = require("fs");
if (fs.existsSync("./docs/blog2.txt")) {
  fs.unlink("./docs/blog2.txt", (err) => {
    if (err) {
      console.log(err);
    }
    console.log("file deleted");
  });
}
```

Create a folder



To create a new folder, we can use the `fs.mkdir()` method. We'll need to specify what folder to make and where.

```
const fs = require("fs");
fs.mkdir("./images", (err) => {
  if (err) {
    console.log(err);
  }
  console.log("folder created");
});
```

Check if a folder exists

We can check whether a folder exists before creating it with the `fs.existsSync()` method.

```
const fs = require("fs");
if (!fs.existsSync("./images")) {
  fs.mkdir("./images", (err) => {
    if (err) throw err;
    console.log("folder created");
  });
}
```

Remove a folder

```
const fs = require("fs");

fs.rmdir("./images", (err) => {
  if (err) {
    console.log(err);
  }
  console.log("folder deleted");
});
```

Stream

A large file can take a long time to read, so we'll waste time waiting. To avoid that, we can use Streams.

Using stream, we can start using the data before it has finished loading.

1. Read stream

To open a file as a readable stream, we can use `fs.createReadStream()`. It takes two arguments:

- The first argument is the path of where we want to read and pass data from
- The second argument is an object `{ encoding: "utf8" }` so the stream will be in a readable format.

We'll then create an event listener whenever we create a chunk of data:

```
const fs = require("fs");
const readStream = fs.createReadStream("./docs/blog3.txt", {
  encoding: "utf8",
```



```
});  
readStream.on("data", (chunk) => {  
  console.log("--New Chunk--");  
  console.log(chunk);  
});
```

2. Write stream

We can use `fs.createReadStream()` to write data to a file a bit at a time.

For example, we'll pass data of `blog3` to `blog4`:

```
const fs = require("fs");  
const readStream = fs.createReadStream("./docs/blog3.txt", {  
  encoding: "utf8",  
});  
const writeStream = fs.createWriteStream("./docs/blog4.txt");  
readStream.on("data", (chunk) => {  
  writeStream.write(chunk);  
});
```

What is an OS module?

The OS module provides information about the computer's operating system.

To include the OS module, we use the `require()` method:

```
const os = require("os");
```

OS Properties and Methods

Method	Description
<code>arch()</code>	Returns the operating system CPU architecture
<code>constants</code>	Returns an object containing the operating system's constants
<code>cpus()</code>	Returns an array containing information about the computer's CPUs
<code>endianness()</code>	Returns the endianness of the CPU
<code>EOL</code>	Returns the end-of-line marker for the current operating system
<code>freemem()</code>	Returns the number of free memory of the system
<code>hostname()</code>	Returns the hostname of the operating system
<code>loadavg()</code>	Returns an array containing the load averages (1, 5, and 15 minutes)
<code>networkInterfaces()</code>	Returns the network interfaces that have a network address
<code>platform()</code>	Returns information about the operating system's platform
<code>release()</code>	Returns information about the operating system's release
<code>tmpdir()</code>	Returns the operating system's default directory for temporary files
<code>totalmem()</code>	Returns the number of the total memory of the system
<code>type()</code>	Returns the name of the operating system
<code>uptime()</code>	Returns the uptime of the operating system in seconds
<code>userInfo()</code>	Returns information about the current user

Example

Name of the Faculty: Gopal Paul

Designation and Department: Assistant Professor, Computational Science

Brainware University, Kolkata



```
// Platform
console.log(os.platform());
// CPU Arch
console.log(os.arch());
// CPU Core Info
console.log(os.cpus());
// Free memory
console.log(os.freemem());
// Total memory
console.log(os.totalmem());
// Home dir
console.log(os.homedir());
// Uptime
console.log(os.uptime());
```

What is the URL module?

The URL module is quite similar to the Path module, but instead of handling the path, this module handles the current URL the user is accessing.

URL String & URL Object

A URL string is made up of many small components, and we can process and retrieve those small ones using the URL module.

```
const url = require("url");
```

Example

```
const newUrl = new URL(
  "https://www.example.com:8000/software/index.html?q=abc"
);
// Serialized URL
console.log(newUrl.href);
console.log(newUrl.toString());
// Host (root domain)
console.log(newUrl.host);
// Hostname (No port)
console.log(newUrl.hostname);
// Port
console.log(newUrl.port);
// Pathname
console.log(newUrl.pathname);
// Serialized query
console.log(newUrl.search);
// Params object
console.log(newUrl.searchParams);
```

Name of the Faculty: Gopal Paul

Designation and Department: Assistant Professor, Computational Science

Brainware University, Kolkata



```
// Add param
newUrl.searchParams.append("status", "alive");
console.log(newUrl.searchParams);
```

2.2.1 Node Local Modules

A local module is a self-contained block of code that can be reused throughout an application. It is a way of organizing your code into small, manageable chunks that can be imported and exported as needed. Local modules allow you to keep your code organized and easy to maintain.

Node.js provides a simple way of creating local modules using the "exports" keyword. When you export a function or an object from a module, you can then import it into other files using the "require()" function.

Let's create a simple local module in Node.js that exports a function called "add". The function takes two parameters and returns their sum. Here is the code for the "add.js" file:

```
const add = (a, b) => {
  return a + b;
};
exports.add = add;
```

In the above code, we define a function called "add" that takes two parameters and returns their sum. We then export this function using the "exports" keyword.

To use this local module in another file, we can use the "require()" function to import it. Here is the code for the "index.js" file:

```
const addModule = require("./add");
console.log("Local Module");
const total = addModule.add(10, 5);
console.log("Total : " + total);
```

In the above code, we use the "require()" function to import the "add.js" module. We then log "Local Module" to the console and call the "add()" function with the values of 10 and 5. We then log the total to the console.

Benefits of Using Local Modules

Using local modules in Node.js offers several benefits:

- **Code organization:** By breaking your code into small, manageable chunks, you can keep your code organized and easy to maintain.
- **Code reusability:** Local modules allow you to reuse code throughout your application, making it easier to scale your application as it grows.
- **Ease of testing:** Local modules can be easily tested in isolation, which makes it easier to identify and fix bugs in your code.

Using Multiple Functions in Local Modules

In addition to exporting a single function or object, you can also export multiple functions or objects from a local module in Node.js. Here is an example of how to create a local module



that exports two functions, "add" and "sub", that perform addition and subtraction respectively.

```
// math.js
const add = (a, b) => {
  return a + b;
};

const sub = (a, b) => {
  return a - b;
};

module.exports = {
  add: add,
  sub: sub,
};
```

In the above code, we define two functions, "add" and "sub", that perform addition and subtraction respectively. We then export these functions using an object that maps the function names to their respective functions.

To use this local module in another file, we can use the "require()" function to import it. Here is the code for the "index.js" file:

```
// index.js
const math = require("./math");

console.log("Local Module");
const sum = math.add(10, 5);
console.log("Sum: " + sum);

const difference = math.sub(10, 5);
console.log("Difference: " + difference);
```

In the above code, we use the "require()" function to import the "math.js" module. We then call the "add()" function with the values of 10 and 5, and log the result to the console. We then call the "sub()" function with the same values, and log the result to the console.

Benefits of Using Multiple Functions in Local Modules

Using multiple functions in local modules can offer several benefits:

- **Code organization:** By grouping related functions into a single module, you can keep your code organized and easy to maintain.
- **Code reusability:** Multiple functions in a local module can be reused throughout your application, making it easier to scale your application as it grows.
- **Simplified syntax:** Using an object to export multiple functions allows you to easily access and use the functions using dot notation.

2.2.3 Node Third Party Modules

Name of the Faculty: Gopal Paul
Designation and Department: Assistant Professor, Computational Science
Brainware University, Kolkata



Third-party Modules (dependencies) in Node.js are pre-written code or packages that is not part of the core Node.js library. These dependencies can be used to add new features to your Node.js application, or to improve the performance or reliability of your application.

There are many different third-party dependencies available for Node.js, and the most popular ones can be found on the **npm** registry. To install a third-party dependency, you can use the **npm install** command. For example, to install the express dependency, you would run the following command:

Example

```
npm install express
```

Once a third-party dependency is installed, you can use it in your Node.js application by importing it into your code. For example, to use the express dependency in your application, you would import it as follows:

```
const express = require('express');
```

Here are some of the most popular third-party dependencies for Node.js:

- 1-Express: A web framework for Node.js
- 2-Mongoose: A MongoDB ODM for Node.js
- 3-Angular: A web application framework for Node.js
- 4-React: A JavaScript library for building user interfaces
- 5- Redux: A state management library for JavaScript

2.3 Overview of the Node Package Manager (NPM)

2.3.1 What is NPM?

NPM – or "Node Package Manager" – is the default package manager for JavaScript's runtime Node.js.

NPM (Node Package Manager) is a package manager for Node.js modules. It helps developers manage project dependencies, scripts, and third-party libraries. By installing Node.js on your system, NPM is automatically installed, and ready to use.

- It is primarily used to manage packages or modules – these are pre-built pieces of code that extend the functionality of your Node.js application.
- The NPM registry hosts millions of free packages that you can download and use in your project.
- NPM is installed automatically when you install Node.js, so you don't need to set it up manually.

2.3.2 Why Use npm?

Here are some of the reasons why you should use npm:

1. It enables you to install libraries, frameworks, and other development tools for your project, similar to installing a mobile application from an app store.
2. You gain access to safe Node.js projects for development.
3. It helps you speed up the development phase by using prebuilt dependencies.
4. npm has a wide variety of tools to choose from for no cost.

Name of the Faculty: Gopal Paul

Designation and Department: Assistant Professor, Computational Science

Brainware University, Kolkata



5. Using npm commands doesn't require a lot of learning, as they're easy to understand and make use of.

2.3.3 Packages vs. Modules: The Building Blocks of npm

- **Packages:** A package typically consists of a collection of code often comprising JavaScript files along with a package.json file. This particular file serves as the package identification card encompassing details such as its name, version number, dependencies, and scripts for project management. Packages range in scope from utility functions to comprehensive web application frameworks such, as React or Angular.
- **Modules:** A module is, like a JavaScript file that offers a particular reusable function. Modules make use of the **require()** function to bring in code from modules and utilize the exports object to showcase their own functionality. They promote organized and easily maintainable code.

2.3.4 Essential npm Commands

- ✓ **npm install:** This command is used to install packages. You can either install packages globally or locally. When a package is installed globally, we can make use of the package's functionality from any directory in our computer.
On the other hand, if we install a package locally, we can only make use of it in the directory where it was installed. So no other folder or file in our computer can use the package.
- ✓ **npm uninstall:** This command is used to uninstall a package.
- ✓ **npm init :** The init command is used to initialize a project. When you run this command, it creates a **package.json** file.
- ✓ When running **npm init**, you'll be asked to provide certain information about the project you're initializing. This information includes the project's name, the license type, the version, and so on.
- ✓ To skip the process of providing the information yourself, you can simply run the **npm init -y** command.
- ✓ **npm update:** Use this command to update an npm package to its latest version.
- ✓ **npm restart :** Used to restart a package. You can use this command when you'd want to stop and restart a particular project.
- ✓ **npm start :**Used to start a package when required.
- ✓ **npm stop :**Used to stop a package from running.
- ✓ **npm version:** Shows you the current npm version installed on your computer.
- ✓ **npm publish:** Used to publish an npm package to the npm registry. This is mostly used when you have created your own package.

2.3.5 How To Install an npm Package Globally

When you install an npm package globally, you're able to access it from any directory on your computer.

In this section, you'll see a practical way of installing a package globally by running an npm command in your terminal.

Name of the Faculty: Gopal Paul

Designation and Department: Assistant Professor, Computational Science

Brainware University, Kolkata



To install a package globally, use this command:

```
npm install -g [package name]
```

Note that the **-g** flag in the command is what enables the npm CLI to install the package globally.

Here is an example:

```
npm install -g nodemon
```

The command above will install nodemon globally on your computer. After the installation, you can use TypeScript in any directory.

How To Install an npm Package Locally

In the previous section, we saw how to install an npm package globally. Now let's talk about installing one locally.

To install a package locally means you can only use the package's functionality in the current directory. To do this, you'd have to navigate to the directory you want to install the package and run this command in the terminal:

```
npm install [package name]
```

Here is an example:

```
npm install express
```

The command above will install express locally, meaning it will only function in this current directory.

2.3.6 How To Uninstall an npm Package Globally

In situations where we no longer need an npm package, you can remove it from your computer by uninstalling it.

To uninstall a package globally, use this:

```
npm uninstall -g [package name]
```

Let's see an example:

```
npm uninstall -g nodemon
```

The command in the example above will remove the nodemon package from your computer.

2.3.7 How To Uninstall an npm Package Locally

Uninstalling a locally installed npm package is similar to the previous example, except this time, we won't be using the **-g** flag.

Here is the syntax:

```
npm uninstall [package name]
```

And here is a working example:

```
npm uninstall express
```

2.3.8 How To Update npm and Packages

Keeping your npm and packages up to date is the single best way to keep bugs and security flaws away from your code.

To update npm to its latest version, use the command below:

```
npm install npm@latest -g
```

Name of the Faculty: Gopal Paul

Designation and Department: Assistant Professor, Computational Science

Brainware University, Kolkata



This updates npm globally on your computer.

When the creators of a package introduce new features or fix bugs, they update the package on the npm registry. You then have to update your own package in order to make use of the new features.

Here is the syntax of the command you'd use to do this:

```
npm update [package name]
```

And here's a working example:

```
npm update express
```

The command above updates express to its latest version.

Additionally, just like in the previous sections, we can use the -g flag to update a package globally. That is:

```
npm update -g typescript
```

2.3.9 How To Change Location of npm Packages

For some users who don't have administrative permissions on their computer, running npm commands may return an error message. To fix this, you can change the default installation location of your packages by setting a new location/directory.

Here is the syntax for doing that:

```
npm config set prefix [new directory path]
```

2.3.10 How To List Installed npm Packages Globally

If you're wondering how to check for the number of packages installed on your computer, npm provides a command that lists them out.

Entering the command below lists all the packages installed globally on your device:

```
npm list -g
```

When the command above is successfully executed, you'll see a full list of packages you've previously installed anywhere on your computer.

2.3.11 How To List Installed npm Packages Locally

Similar to the last section, we can list locally installed npm packages, too.

To see a list of packages installed locally, run the command below in your terminal:

```
npm list
```

You have to run the above command in your project's directory in order to see all the packages installed for that particular project.

2.3.12 The package.json File

The **package.json** file helps us keep track of all the installed packages in a given project. When creating a new project, it is important to start by creating this file.

It stores information about a project like the name of the project, its version, scripts, dependencies, and more.

You can do this by running the **npm init** or **npm init -y** command in the project's terminal. Then just fill in all the questions the system asks when creating the file.

After generating the **package.json** file, all the packages installed, along with their names and versions, will be stored in the file.

Name of the Faculty: Gopal Paul

Designation and Department: Assistant Professor, Computational Science

Brainware University, Kolkata



Another important use of the **package.json** file is seen when we clone projects on GitHub. When developers push their project to a repository, they leave out the `node_modules` folder, which contains our packages and their dependencies.

In order to generate your own folder after cloning a repo, you have to run the ***npm install*** command in your project's terminal. This will enable npm to go through the repository's **package.json** file and install all the packages listed there.

After the installation is complete, you can then make use of all the packages previously installed for that project before it gets pushed to GitHub.

2.3.12.1 Purpose of the package.json File:

- **Dependency Management:** The `package.json` file lists all the dependencies required for the NodeJS project. These dependencies include both runtime dependencies needed for the application to function and development dependencies required for building, testing, and maintaining the project.
- **Version Control Integration:** The `package.json` file typically includes metadata such as the project's name, version, description, author, and license. This information helps developers and users understand the project's purpose, track its version history, and comply with licensing requirements.
- **Script Execution:** NodeJS projects often define custom scripts in the `package.json` file to automate various tasks, such as running tests, starting the development server, building the project, or deploying the application. These scripts simplify common development workflows and promote consistency across different environments.

2.3.12.2 Main Keys in package.json

- ✓ **name:** Specifies the project's name and for organization-specific packages, use scoped packages. I.e. `@yourorg/your-package`
- ✓ **version:** Defines the project's version.
- ✓ **description:** Provides a brief description of the project.
- ✓ **dependencies:** Lists the production dependencies required for the project.
- ✓ **devDependencies:** Specifies dependencies used during development and testing.
- ✓ **scripts:** Contains custom scripts for automating tasks.
- ✓ **keywords:** Identifies keywords relevant to the project.
- ✓ **repository:** Indicates the project's version control repository.
- ✓ **license:** Specifies the project's licensing information.
- ✓ **private:** For private packages, set the "private" property to true.

An example `package.json` might look like this:

```
{
  "name": "my-project",
  "version": "1.5.0",
  "description": "Express server project using compression",
  "main": "src/index.js",
  "scripts": {
    "start": "node index.js",
```

Name of the Faculty: Gopal Paul

Designation and Department: Assistant Professor, Computational Science

Brainware University, Kolkata



```
        "dev": "nodemon",
        "lint": "eslint **/*.js"
    },
    "dependencies": {
        "express": "^4.16.4",
        "compression": "~1.7.4"
    },
    "devDependencies": {
        "eslint": "^5.16.0",
        "nodemon": "^1.18.11"
    },
    "repository": {
        "type": "git",
        "url": "https://github.com/osiolabs/example.git"
    },
    "author": "Jon Church",
    "contributors": [{
        "name": "Amber Matz",
        "email": "example@example.com",
        "url": "https://www.osiolabs.com/#team"
    }],
    "keywords": ["server", "osiolabs", "express", "compression"]
}
```

name

```
"name": "my-project"
```

The name field defines the name of the package. When publishing to the NPM registry, this is the name the package will be listed under. It must be no more than 214 characters, only lowercase letters, and it must be URL-safe (hyphens and underscores allowed, but no spaces or other characters disallowed in URLs).

If publishing your package to NPM, the name property is required and must be unique. You'll receive an error if trying to publish a package under a name that is currently used on the NPM registry. If you aren't developing a package which you'll eventually publish, the name does not have to be unique.

version

```
"version": "1.5.0",
```

The version field is very important for any published package, and required before publishing. It is the current version of the software that the *package.json* is describing.

license

This is a very important but often overlooked property. The license field lets us define what license applies to the code the *package.json* is describing. Again, this is very important when

Name of the Faculty: Gopal Paul

Designation and Department: Assistant Professor, Computational Science

Brainware University, Kolkata



publishing a project to the NPM registry, as the license may limit the use of your software by some developers or organizations. Having a clear license in place helps clearly define what terms the software is able to be used under.

The value of this field will usually be the license's identifier code -- a string like "MIT" or "ISC" for the MIT license and ISC license respectively. If you don't wish to provide a license, or explicitly do not want to grant use of a private or unpublished package, you can put "UNLICENSED" as the license. Choose a License is a helpful resource if you're not sure which license to use.

author and contributors

```
"author": "Jon Church jon@example.com https://www.osiolabs.com/#team",  
"contributors": [{  
  "name": "Amber Matz",  
  "email": "example@example.com",  
  "url": "https://www.osiolabs.com/#team"  
}],
```

The author and contributor fields function similarly. They are both "people" fields which can be either a string in the format of "Name <email> <url>" , or an object with fields name, email, url. The email and url are both optional.

Author is for a single person, and contributors is an array of people.

These fields are a useful way to list contacts for a public project, as well as share credit with contributors.

description

The description field is used by the NPM registry for published packages, to describe the package in search results and on the npmjs.com website.

This string is used to help surface packages when users search the NPM registry. This should be a short summary of what the package is for.

It can also be useful as simple documentation for your project, even if you aren't publishing it to the NPM registry.

keywords

```
"keywords": ["server", "osiolabs", "express", "compression"]
```

The keywords field is an array of strings, and serves a similar purpose to the description. This field is indexed by the NPM registry to help find packages when someone searches for them. Each value in the array is one keyword associated with your package.

This field doesn't have much use if you're not publishing to the NPM registry, and you can feel free to omit it.

main

```
"main": "src/index.js",
```

The main field is a functional property of your *package.json*. This defines the entry point into your project, and commonly the file used to start the project.



If your package (let's say its name is `foo-lib`) is installed by a user, then when a user does `require('foo-lib')`, it is the `module.exports` property of the file listed in the `main` field that is returned by `require`.

This is commonly an `index.js` file in the root of your project, but it can be any file you choose to use as the main entry-point to your package.

scripts

```
"scripts": {  
  "start": "node index.js",  
  "dev": "nodemon"  
}
```

The `scripts` field is another functional piece of metadata in your `package.json`. The `scripts` property takes an object with its keys being scripts we can run with `npm run <scriptName>`, and the value is the actual command which is run. These are typically terminal commands, which we put into the `scripts` field so we can both document them and reuse them easily. Scripts are powerful tools that the npm CLI can use to run tasks for your project. They can do the job of most task runners used during development. Learn more about npm scripts.

repository

```
"repository": {  
  "type": "git",  
  "url": "https://github.com/osiolabs/example.git"  
}
```

You can record the repository the code for a project lives in by providing the `repository` field. This field is an object which defines the url where the source code is located, and what type of version control system it uses. For open source projects, this is likely GitHub or Bitbucket with Git as the version control system.

An important note is that the `URL` field is meant to point to where the version control can be accessed from, not just the released code base.

dependencies

```
"dependencies": {  
  "express": "^4.16.4",  
  "compression": "~1.7.4"  
}
```

This is one of the most important fields in your `package.json`, and likely the entire reason you need one. All of the dependencies your project uses (the external code that the project relies on) are listed here. When a package is installed using the npm CLI, it is downloaded to your `node_modules/` folder and an entry is added to your `dependencies` property, noting the name of the package and the installed version.

The `dependency` field is an object with package names as keys, and a version or version range as a value. From this list, npm knows what packages to fetch and install (and at what versions)



when npm install is run in the directory. The dependency field of your *package.json* is at the heart of your project, and defines the external packages your project requires.

The carets (^) and tildes (~) you see in the dependency versions are notation for version ranges defined in SemVer.

devDependencies

```
"devDependencies": {  
  "nodemon": "^1.18.11"  
}
```

Similar to the dependencies field, but for packages which are only needed during development, and aren't needed in production.

An example would be using a tool to reload your project during development, like nodemon, which we have no use for once the application is deployed and in production. The devDependencies property lets us explicitly note which dependencies aren't needed in production. When installing your app in a production environment, you can use npm install --production to only install what is listed in the dependency field of *package.json*.

Recording a *devDependency* is a great way to document what tools are needed for the app during development. To install a package from npm as a devDependency, you can run npm **install --save-dev <package>**.

There is another way the devDependencies property is useful to us, and that's by using them in our npm scripts.